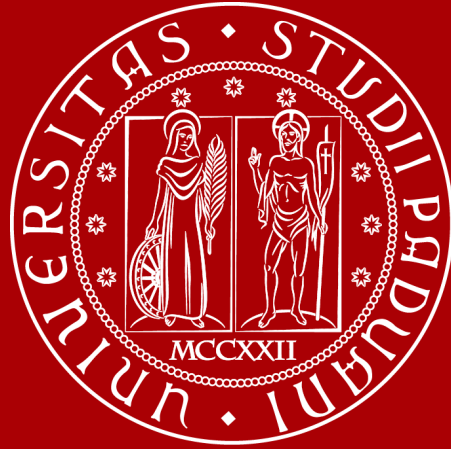




UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Streaming Processing of the QUAX Experiment with Dask and Kafka

Fabio Cimino, Daniele Ghezzi - Group 11



Introduction

QUAX Experiment — Dataset Overview

Resonant RF cavity data for axion dark matter search

Signal components

In-phase (I) and Quadrature (Q) form a complex time-domain signal $I + jQ$

FFT resolution

2048 bins, spanning a 2 MHz bandwidth

Sample encoding

32-bit floating point, little-endian

Target throughput

16 MB/s (realistic DAQ production rate)

Scan size

$2^{11} = 2048$ samples per scan

Raw file format

2 sets of .dat binary files (I and Q),
continuous stream of ADC readings

File size

2^{23} samples per file $\rightarrow$ 32 MB

Production rate

~ 1 file-pair (I + Q) every ~ 4 seconds

Highly parallelizable:
each scan is processed independently via FFT and reduced by averaging

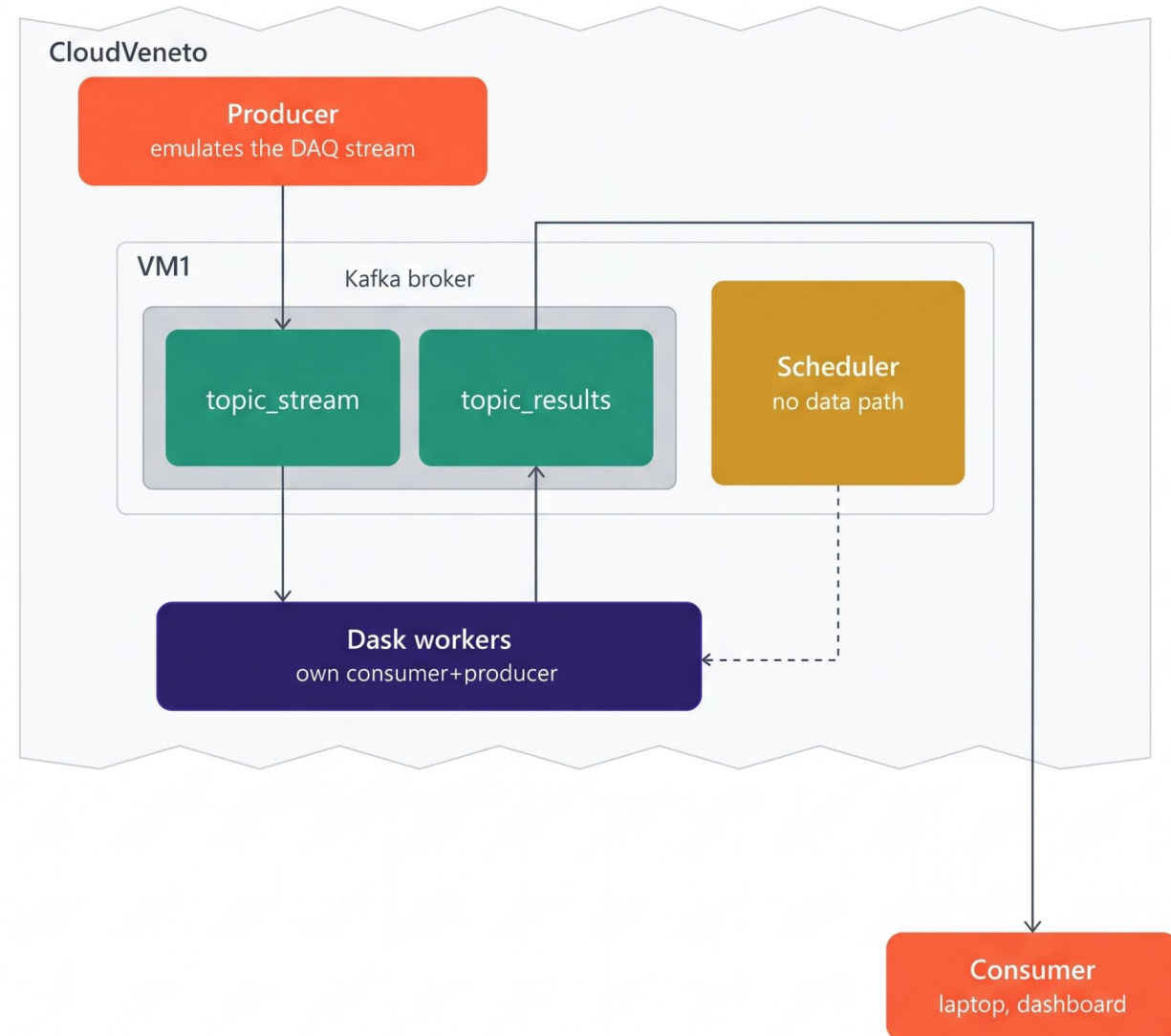
Real time ETL:
enables live monitoring of data collection, and overcomes the DAQ's limited local storage for high rate and long running datasets



Architecture Overview

The architecture exploits **Dask** and **Apache Kafka**

- **DAQ emulator:**
continuously streams scan batches to Kafka topic_stream
- **Kafka broker:**
it buffers raw and processed data on topic_stream and topic_results
- **Dask cluster:**
the scheduler VM also hosts the Kafka broker, while streamed and processed data flows directly through the worker nodes
- **Dashboard:**
renders a live view of the workers and cumulative averaged spectra





DAQ Emulator

Specifications

- VM: MAPD-group11-4
- IP address: 10.67.22.236
- Flavor: cloudveneto.medium (2vCPU, 4GB RAM)

Operation

- Loads the entire dataset (~2 GB) into memory
- Computes the publish rate from **configurable batch size** (n. of scans per batch) and **throughput**
- Publishes data continuously to topic_stream at the configured rate
- Sends begin and end signals to the dashboard (explained later)



Maintaining the throughput

```
start_time = time.perf_counter()

for i in range(N_BATCHES):
    # Calculate slicing indices for the current batch
    start = i * BATCH_SIZE_B//2
    end = start + BATCH_SIZE_B//2

    # Concatenate I and Q into a single packet
    batch[:BATCH_SIZE_B//2] = i_data[start:end]
    batch[BATCH_SIZE_B//2:] = q_data[start:end]

    # Send packet to Kafka and measure the time for sending
    producer.send(TOPIC_STREAM, value=batch)
    producer.flush()

    # Calculate the target time and sleep if necessary to maintain the target publish interval
    target_time = start_time + (i + 1) * TARGET_PUBLISH_INTERVAL_S
    sleep_time = target_time - time.perf_counter()

    if sleep_time > 0:
        time.sleep(sleep_time)
    else:
        print(f"missed send by {sleep_time}s")
```



DAQ emulator location

One issue with the throughput

```
dghezzi@PcDaniele:~/Project_MAPD$ iperf3 -c localhost -p 15201
Connecting to host localhost, port 15201
local 127.0.0.1 port 60206 connected to 127.0.0.1 port 15201
```

```
....
Interval          Transfer      Bitrate      Retr
0.00-10.01  sec   139 MBytes  117 Mbits/sec   16
0.00-9.76   sec   131 MBytes  113 Mbits/sec

sender receiver  ≈ 14 MB/s
```

```
debian@mapd-group11-4:~$ iperf3 -c 10.67.22.90
Connecting to host 10.67.22.90, port 5201
local 10.67.22.236 port 60000 connected to 10.67.22.90 port 5201
```

```
.....
Interval          Transfer      Bitrate      Retr
0.00-10.00  sec   4.05 GBytes  3.48 Gbits/sec  1120
0.00-10.00  sec   4.05 GBytes  3.48 Gbits/sec

sender receiver  ≈ 400 MB/s
```

-> The DAQ emulators needs to be placed inside CloudVeneto network



Kafka Broker

Specifications

- VM: MAPD-group11-1
- IP address: 10.67.22.90
- Flavor: cloudveneto.medium (2vCPU, 4GB RAM)

Operation

- Handles producer/consumer connections and message delivery
- Listens on port 19092 for internal network communication, port 9092 (forwarded via ssh) for external communications
- Stores stream logs in a ramdisk and manages their cleanup



Broker network settings

```
#####  
# Broker setup  
#####  
  
...  
  
# The largest message size allowed by the broker. We set this to 65MB  
message.max.bytes=68157440  
  
#####  
# Network configuration  
#####  
# Listener name, hostname and port advertised to clients. INTERNAL is for to cloud VMs and EXTERNAL is for laptops  
advertised.listeners=INTERNAL://10.67.22.90:19092,EXTERNAL://localhost:9092  
  
...  
  
# The maximum size of a request that the socket will accept: 65MB  
socket.request.max.bytes=68157440  
  
# TCP receive buffer size used by broker sockets. 65MB  
socket.receive.buffer.bytes=68157440  
  
# TCP send buffer size used by broker sockets. 1MB  
socket.send.buffer.bytes=1048576
```




RAM usage and retention policies

```
#####  
# Log storage / retention configuration  
#####  
# A comma separated list of directories under which to store log files. /tmp is mounted as tmpfs  
log.dirs=/tmp/kraft-combined-logs  
  
# The maximum time after which a log segment is closed. Closed segments are eligible for deletion according to log.retention policies  
log.roll.ms=1000  
  
# The maximum size of a log segment file. When this size is reached a new log segment will be created. We set this to 65MB  
log.segment.bytes=68157440  
  
# Delete log segments older than this retention period. Applies to closed segments  
log.retention.minutes=3  
  
# A size-based retention policy for logs. Functions independently of log.retention.hours. We set this to the size of 6 files (192MB)  
log.retention.bytes=201326592  
  
# The frequency that the log cleaner checks whether any log is eligible for deletion  
log.retention.check.interval.ms=10000  
  
# The amount of time to wait before deleting a segment scheduled for deletion  
log.segment.delete.delay.ms=1000
```



Dask Workers

Specifications

Worker ID	VM name	VM IP	CPU model name
1	MAPD-group11-2	10.67.22.58	Intel(R) Xeon(R) Gold 6238R
2	MAPD-group11-3	10.67.22.244	AMD EPYC 7413
3	MAPD-group11-5	10.67.22.210	Intel(R) Xeon(R) Gold 6238R
4	MAPD-group11-6	10.67.22.57	Intel(R) Xeon(R) Gold 6238R

- Flavor: cloudveneto.medium (2vCPU, 4GB RAM)

Operation

- Each worker owns its own Kafka producer and consumer
- Consumers subscribe to topic_stream in the same consumer group
- Producers publish data to topic_result; Kafka handles results batching



Dask Workers

```
def worker_function(worker_id, bootstrap_servers, input_topic, output_topic):

    consumer = KafkaConsumer(
        ...
        group_id = 'dask_processing_group', # consumer group to join for dynamic partition assignment
        ...
    )
    producer = KafkaProducer(
        ...
        linger_ms = 5000, # Kafka batching time
        ...
    )
    for msg in consumer:
        ...
        i_data = np.frombuffer(msg.value[:batch_size//2], dtype="<f4") # < = little-endian, f4 = float32
        q_data = np.frombuffer(msg.value[batch_size//2:], dtype="<f4")
        ...

        matrix = complex_signal.reshape((n_scans, N_SAMPLES_PER_SCAN))
        spectra = np.fft.fft(matrix, axis=1)
        power_spectra = np.abs(spectra) ** 2
        ...
        result = {
            "worker_id": worker_id,
            "results": {
                "n_scans": n_scans,
                "power_means": power_means.tolist(),
                "power_M2s": power_M2s.tolist(),
                "producer_ts": msg.timestamp/1000,
                "processing_time" : processing_time,
            }
        }
        producer.send(output_topic, value=result)
```



Dashboard

Operation

- Runs locally on our laptop
- Receives begin and end signals from the DAQ to reset plots, measure timings, ...
- Plots global plot and last update from each worker
- Saves all useful results for benchmark plots



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Live Demonstration



Test Parameters and Metrics

Parameters:

- Throughput: fixed at 16 MB/s
- N. workers: 3,4
- N. topic_stream partitions: 1,2 times the number of workers
- N. of scans per batch: 512, 1024, 2048, 4096
- At least 3 runs for each configuration

Metrics:

- DAQ emulator and dashboard timestamps
- Batch wall time + Batch processing (FFT) time
- Total wall time = analysis_end_ts - producer_begin_ts
- Achieved throughput = (total scans * dimension scan) / Total wall time

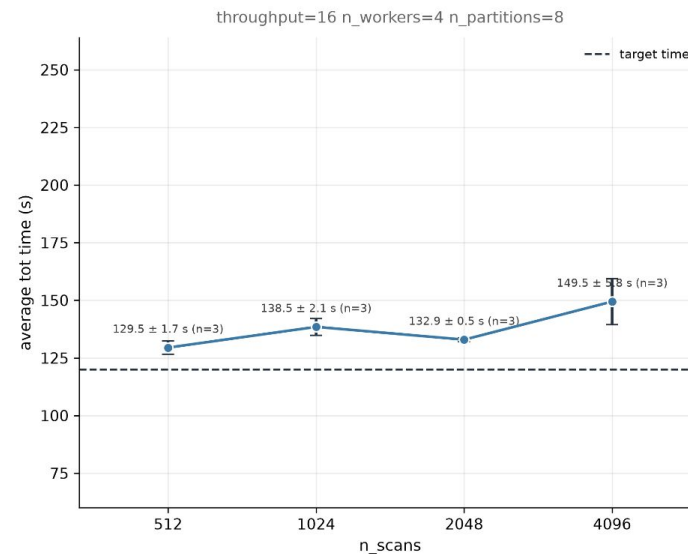
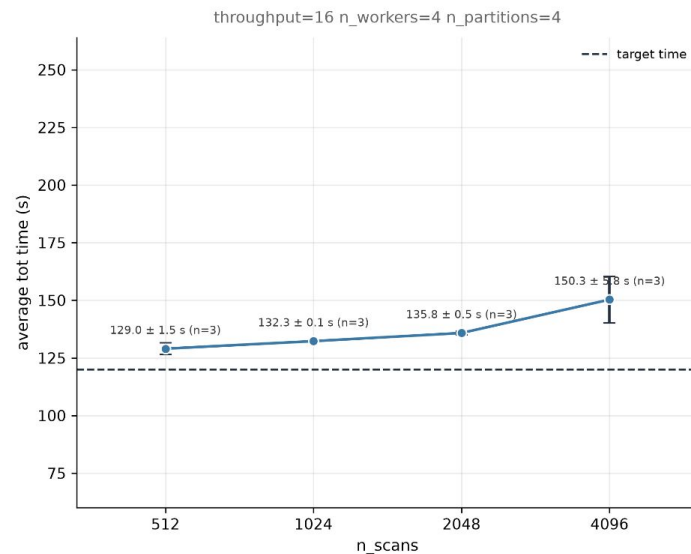
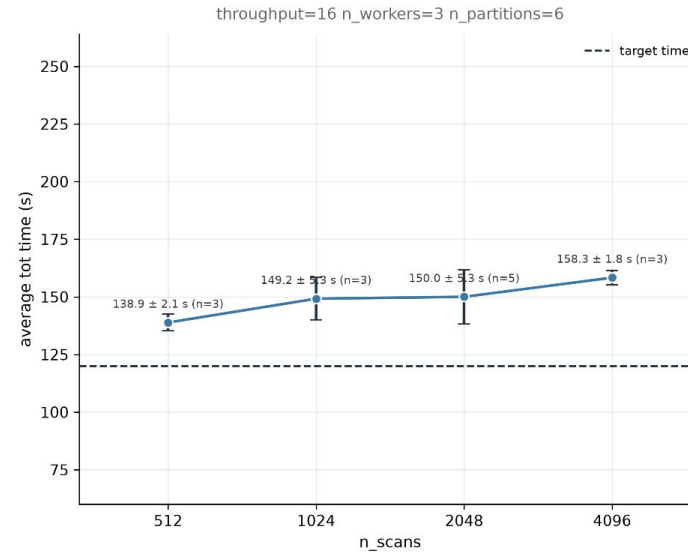
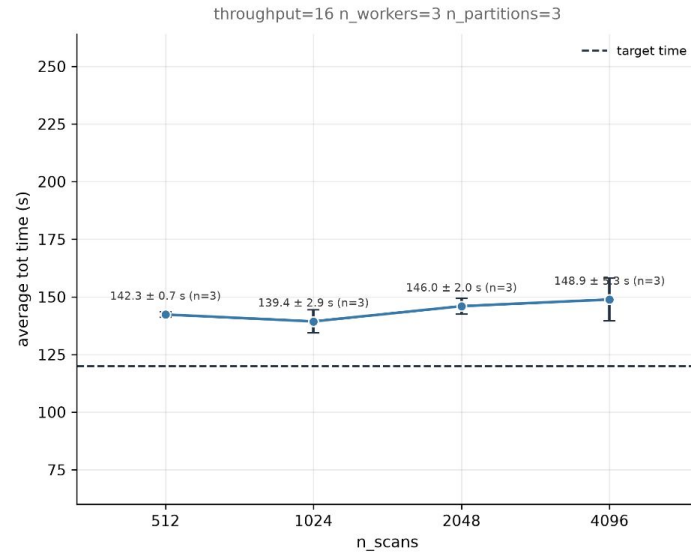
```
{
  "throughput_MB": 16,
  "n_workers": 4,
  "n_partitions": 8,
  "n_total_scans": 126976,
  "n_scans_per_batch": 512,
  "producer_begin_ts": 1784115488.049,
  "producer_end_ts": 1784115613.472,
  "analysis_end_ts": 1784115619.4359076,
  "records": [
    {
      "worker_id": 2,
      "result_ts": 1784115267.18,
      "receive_ts": 1784115268.0165532,
      "n_scans": 512,
      "production_ts": 1784115265.954,
      "batch_latency_ms": 2062.5531673431396,
      "processing_time_ms": 24.07952399516944
    },
    ...
  ]
}
```

*A single data run is saved in
JSON file*



Benchmarks

Total Time vs n_scans_per_batch

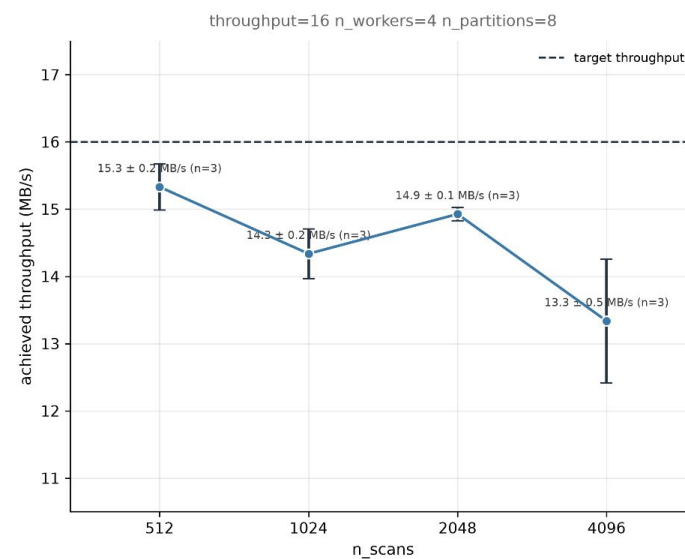
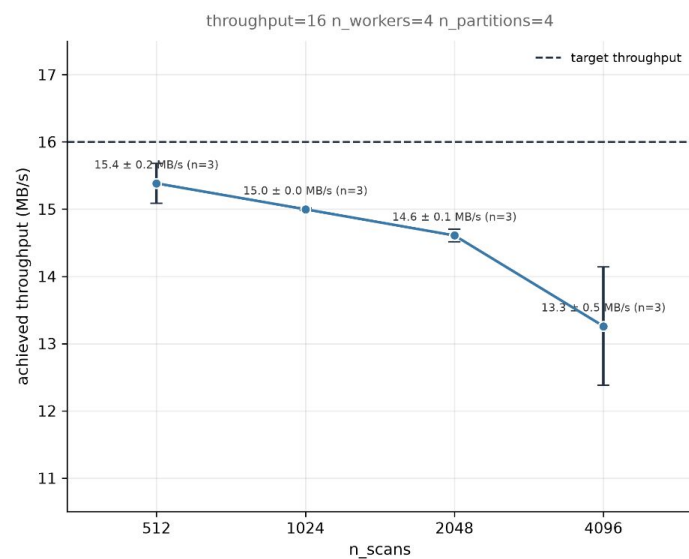
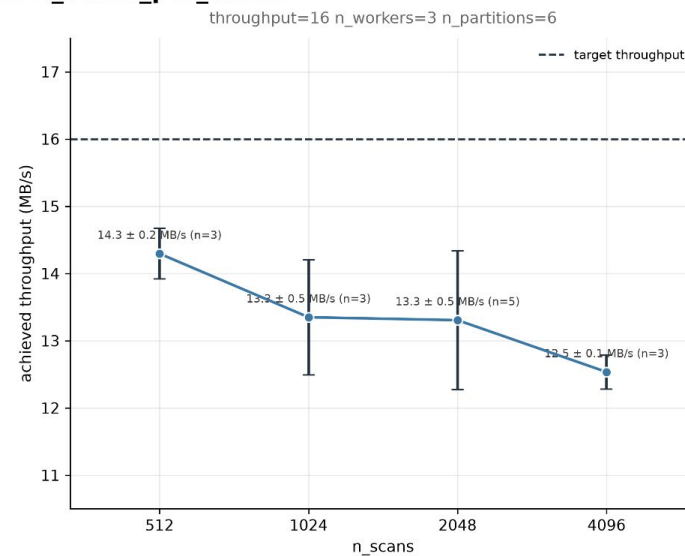
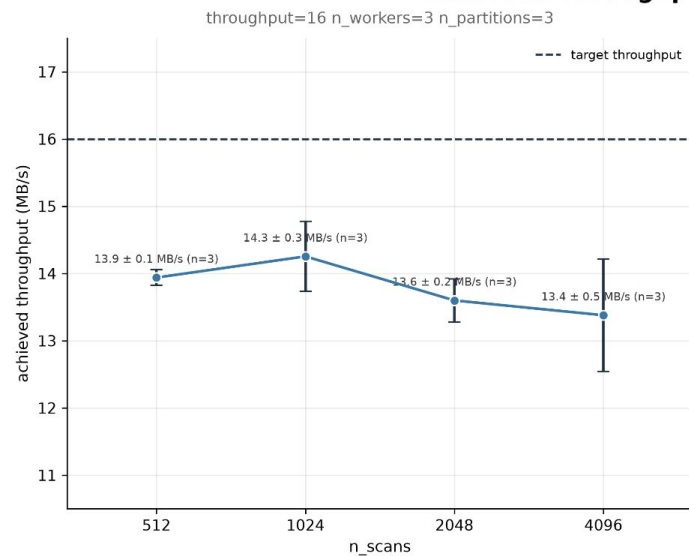


**Total Wall Time vs
number of scans per
batch**



Benchmarks

Achieved Throughput (MB/s) vs n_scans_per_batch



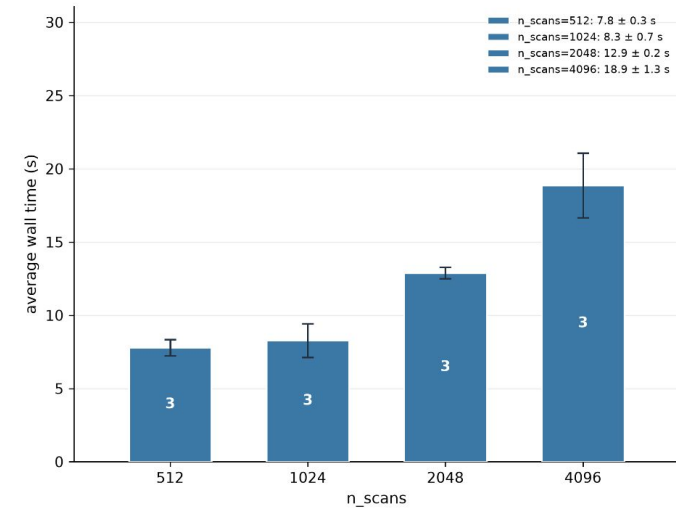
**Achieved Throughput
vs number of scans per
batch**



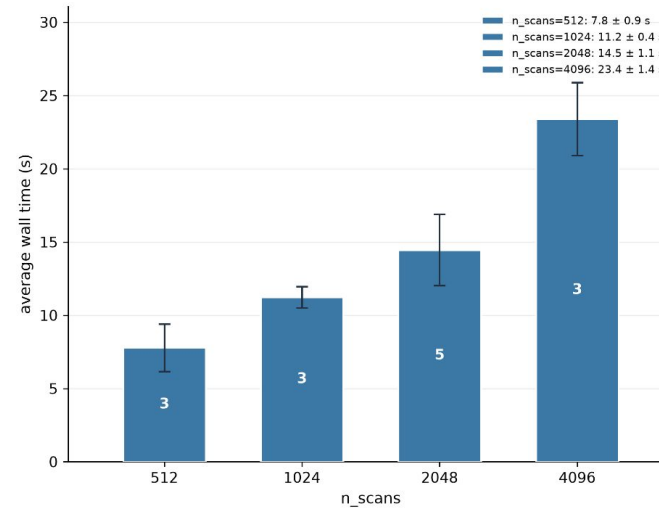
Benchmarks

Avg Wall time (s) vs n_scans_per_batch

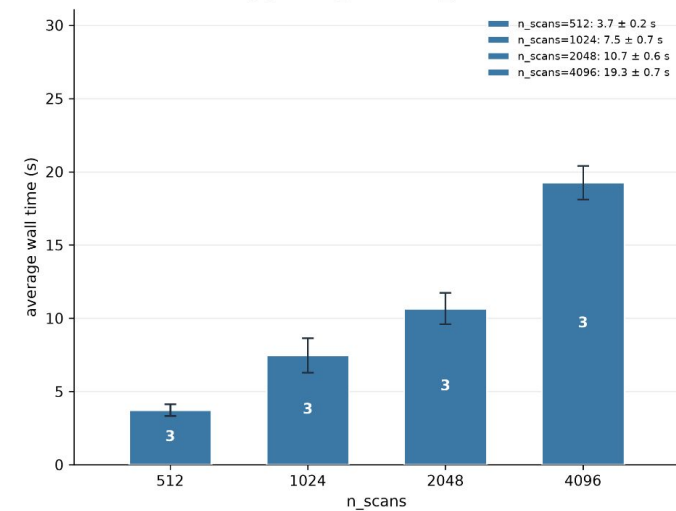
throughput=16 n_workers=3 n_partitions=3



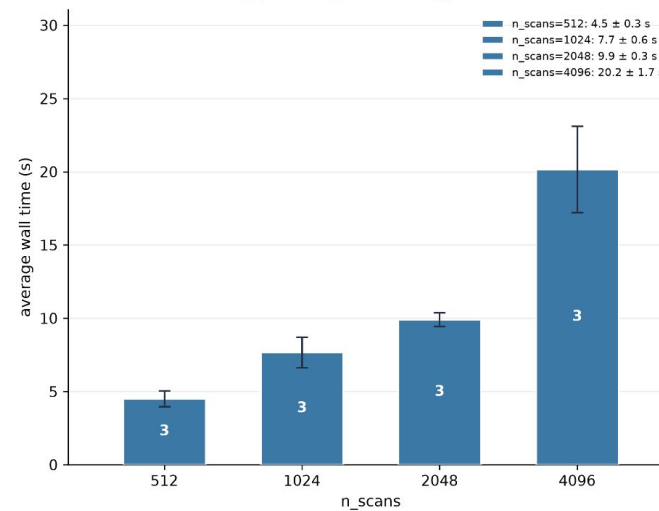
throughput=16 n_workers=3 n_partitions=6



throughput=16 n_workers=4 n_partitions=4

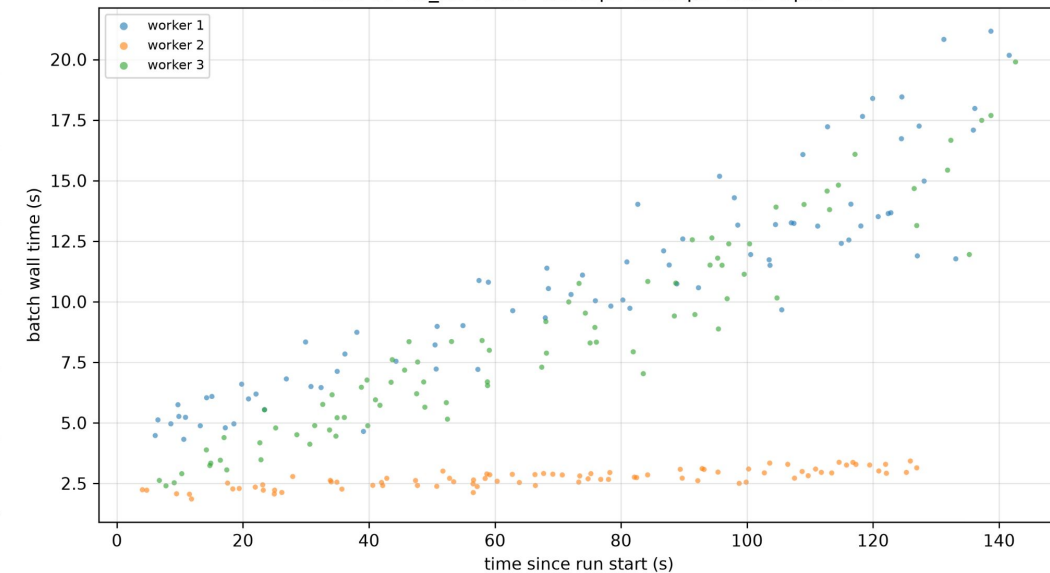


throughput=16 n_workers=4 n_partitions=8



Average Batch Wall time

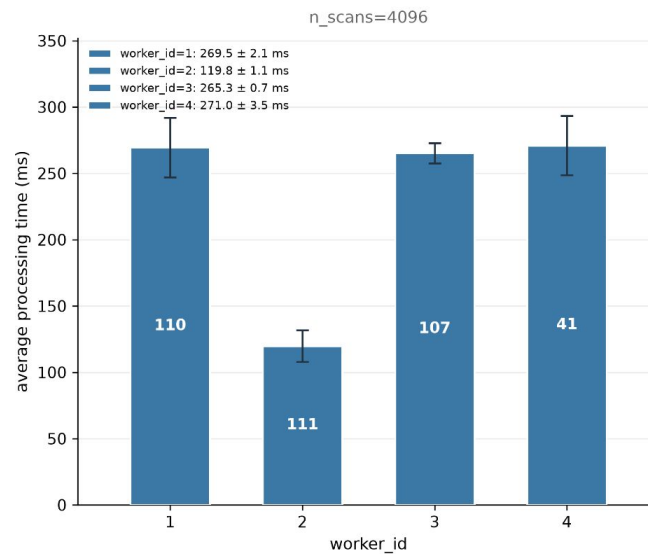
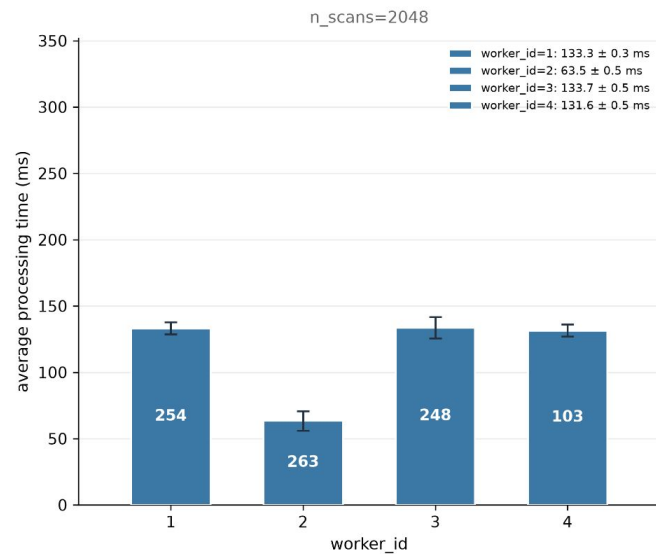
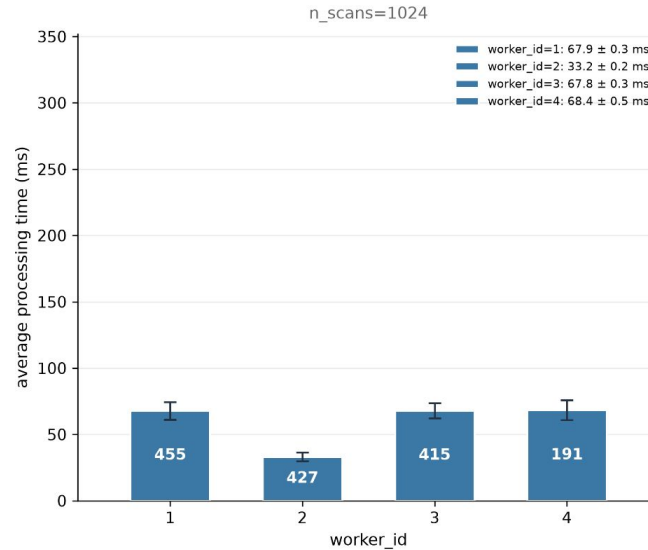
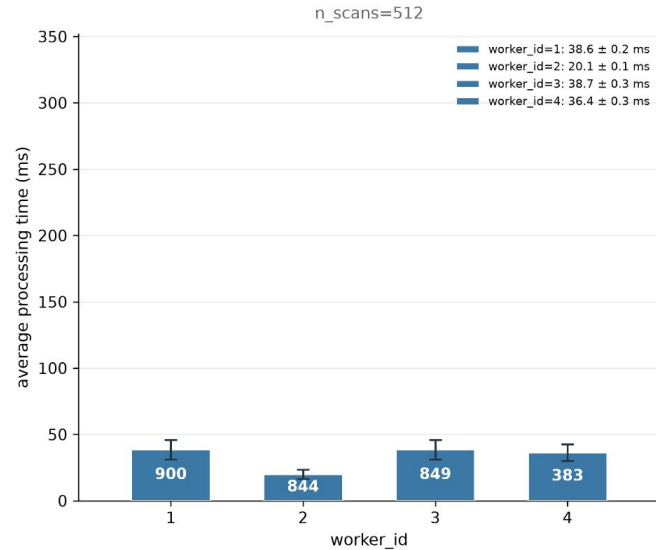
Batch Wall Time vs Time - 16MBps_512SpB_3ws_3parts
2026-07-15_21-37-50--16MBps--512SpB--3ws--3parts





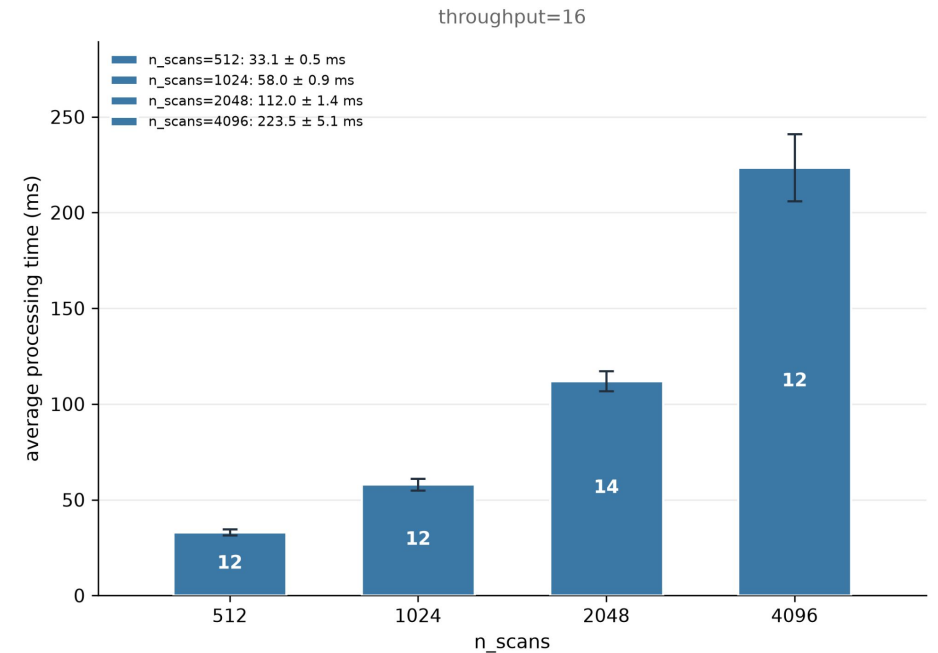
Benchmarks

Avg Processing Time (ms) by Worker



Average Processing time

Avg Processing Time (ms) vs n_scans_per_batch





Encountered Issues

kafka-python sender bug breaking DAQ with large messages:

Version 3.0.4 had a bug that made sending large messages fail silently. We found version 3.0.8 (published on July 9th) fixed the issue.

kafka-python checksum issue affecting DAQ throughput:

kafka-python computes CRC32 checksum for each message using a very slow, pure python implementation. We found that installing library `crc32c` replaces this implementation with an optimized one solving the throughput bottleneck.

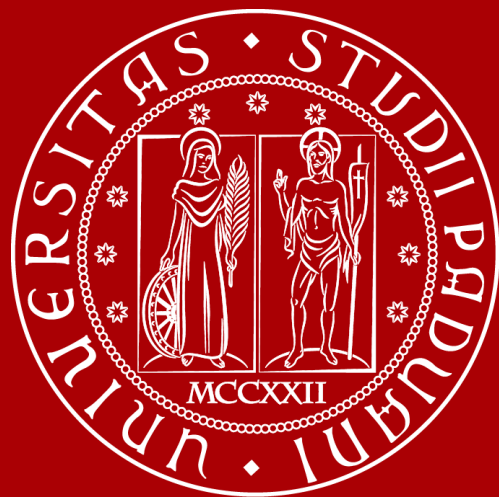
Kafka broker running out of memory:

Kafka broker VM only has 4GB of RAM. After a first run, the JVM keeps 2GB always reserved. The retention policies need to be tuned to minimize memory usage, especially when using a large number of partitions.



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Thank you for your attention!



UNIVERSITÀ
DEGLI STUDI
DI PADOVA